Assignment 2: Strings and Collections

Programming in Java | CO2 | Predict - Measure - Analyse - Redesign

STUDENT	REPLACE_WITH_STUDENT_NAME
REGISTER NUMBER	192511019
JAVA VERSION	Microsoft OpenJDK 17.0.20
GITHUB	github.com/Emadfar007/CSA0915-192511019/tree/main/Assignment%202

Repository-ready report with repeated measurements, raw data, correctness tests and a validated redesign.

Experimental method

Predictions were written from the data-structure contracts before interpreting results. Each benchmark performs warm-up work and repeated timed runs using `System.nanoTime()`, consumes results through a volatile sink, and reports averages. Measurements were made on an AMD Ryzen 5 5600H (12 logical processors), Windows NT 10.0.26300.0 and Microsoft OpenJDK 17.0.20. JVM compilation, garbage collection, CPU frequency and other activity introduce noise; therefore operation counts and growth trends are primary evidence, while nanosecond values describe this machine only. Complete CSV/text evidence is retained under `raw-data`.

Q1 - Building text under load

Prediction and design

`String` is immutable. An append in a loop creates a new result and copies the previous prefix, so appending n similarly sized fields approaches `O(n^2)` copied characters and many temporary objects. `StringBuilder` and `StringBuffer` use growable arrays and provide amortised `O(n)` construction; Buffer synchronizes its operations. Workloads were deliberately contrasted: one ticket with 10 fields, a server batch of 1,000 tickets with 50 fields, and 200 unusually wide tickets with 200 fields.

Workload	Method	Average ns	Average retained-run memory delta	Estimated intermediate objects
1 x 10	String +	99,580	33,593	30
1 x 10	Builder	31,440	41,984	2
1 x 10	Buffer	31,960	41,992	2
1,000 x 50	String +	3,563,000	6,297,249	150,000
1,000 x 50	Builder	3,056,400	1,363,736	2,000
1,000 x 50	Buffer	4,739,460	1,358,936	2,000
200 x 200	String +	13,147,440	20,979,932	120,000
200 x 200	Builder	2,593,200	2,034,561	400
200 x 200	Buffer	3,616,780	2,022,376	400

The prediction is supported most strongly by the wide case: Builder was about 5.1 times faster than repeated + and the estimated temporary count fell from 120,000 to 400. The short case is too small for a meaningful memory conclusion.

Recommendation: use a thread-confined `StringBuilder` per ticket. A truly shared mutable log needs synchronization, so `StringBuffer` is safe; a production logging framework or queue with one writer is preferable. `String.join`/`StringJoiner` can remain linear; `String.format` is readable but has parsing/formatting cost. Repeated loop concatenation remains the only quadratic alternative.

Q2 - ArrayList as an engineering choice

Prediction: ArrayList append is amortised `O(1)`, indexed get `O(1)`, and front/middle insertion `O(n)` due to shifting. LinkedList end/front operations are `O(1)` but indexed traversal is `O(n)`. Three sizes tested the boundary.

Size	List	Front churn ns	Middle churn ns	Middle get ns
1,000	ArrayList	2,215	878	149
1,000	LinkedList	990	2,956	1,147
10,000	ArrayList	2,616	1,406	141
10,000	LinkedList	218	20,863	10,403
50,000	ArrayList	13,447	6,607	135
50,000	LinkedList	218	105,038	53,649

ArrayList is appropriate for append/read/random-access playlists and cache-friendly traversal. Stop recommending it when frequent front shifts dominate. LinkedList is useful at the front or with an already-positioned iterator, but not for repeated indexed middle access. For the changed Up Next workflow, ArrayDeque supports both ends in amortised `O(1)` and measured 15-129 ns per end operation. A plain array duplicates ArrayList shifting/resizing work manually.

Q3 - Engineering the hashCode-equals contract

Three predictions were tested. Adding the same broken reference twice appears correct (predicted/actual size 1). Adding two distinct equal registration objects without hashCode exposes the defect (predicted/actual size 2 in this run). Mutating a registration used in hashCode after insertion makes `contains` false because the object remains in its old bucket.

HashSet first selects a bucket from hashCode and then calls equals among bucket candidates. Therefore equal objects must have equal hashes, and hashed identity fields must be immutable. TODO 1 uses `regNo.hashCode()`; the completed program prints `Unique students: 2`. Before TODO 1, the sample typically reports 3, but the broken program's exact result is not a portable guarantee because identity hashes may collide.

TreeSet with a consistent registration comparator performs `O(log n)` scans and keeps order. HashMap keyed by registration makes identity explicit and gives average `O(1)` scans. Each uses `O(n)` space; HashSet is simplest when only membership is required.

Q4 - Choosing a map for pharmacy inventory

HashMap offers average `O(1)` put/get but no report order. TreeMap offers `O(log n)` operations and natural code order. LinkedHashMap offers average `O(1)` operations plus insertion order. At 10,000 medicines, lookup averages were 50, 285 and 85 ns respectively - all far below the defined 1 ms interactive threshold.

Medicines	Map	Lookup ns	Code report ns	Natural required order
100	HashMap	85	465,080	neither
100	TreeMap	144	72,780	code
100	LinkedHashMap	32	104,060	insertion
1,000	HashMap	37	1,171,520	neither
1,000	TreeMap	178	164,620	code
1,000	LinkedHashMap	37	306,160	insertion
10,000	HashMap	50	5,256,240	neither
10,000	TreeMap	285	681,380	code
10,000	LinkedHashMap	85	1,057,460	insertion

For lookup-heavy operation, use a hash index and maintain ordered report views if reports are frequent. Hashtable is not preferred: it is legacy, synchronizes every operation, forbids nulls, and lacks modern composition/concurrency choices. A sorted array gives compact `O(n)` storage, `O(log n)` lookup, `O(n)` insertion and a free code-order report; it suits read-only snapshots.

Q5 - Generic ranking from a consistency principle

Separate comparators are chosen because Team may have several valid views. The comparator chain is points descending, goal difference descending, then name ascending. `Leaderboard<T>` accepts `Comparator<? super T>`, `addAll(List<? extends T>)`, and returns a sorted copy.

Required output: `[Eagles(9,6), Bisons(9,4), Panthers(9,4), Cobras(6,1)]`. Bisons and Panthers tie on numeric keys; name A-Z resolves the order. The chain is transitive because it lexicographically composes transitive comparisons and returns equality only when all compared keys tie. An intentionally faulty subtraction comparator

overflowed: it returned `1` for MAX versus MIN, while the correct comparator returned `-1`.

One chained sort is `O(n log n)` with `O(n)` copy space. Three stable sorts remain `O(n log n)` asymptotically but perform three passes and require least-significant-first discipline. Comparable also takes one sort but permanently couples one ranking to Team; Comparator is easier to reuse for players and colleges.

Q6 - Priority, order and fairness

A FIFO Queue cannot select a later critical patient. PriorityQueue compares severity ascending and then a monotonically increasing arrival counter; admission and removal are `O(log n)`, and ties remain FIFO. Printing the heap directly does not show total treatment order; a copied heap must be polled.

The eight-patient hand order `P1 P3 P5 P7 P0 P6 P2 P4` matched the program. Growth measurements at 1,000, 5,000 and 20,000 patients remained below 2 microseconds per operation after warm-up. The design is more than adequate for 5,000 daily arrivals; persistence, concurrency and clinical policy are the larger deployment issues.

Severity 5 can starve if higher priorities keep arriving. Proposed aging: after each 30 waiting minutes, improve effective severity by one level, capped at 1, and rebuild/re-evaluate before selection. This bounds waiting but adds periodic `O(n)` recalculation and requires clinical approval. A sorted LinkedList has `O(n)` admission and `O(1)` serve; five FIFO severity queues give specialised `O(1)` operations and are a strong fixed-range alternative.

Q7 - Diagnose, redesign and validate

A-B. Diagnosis and measurable objective

Q4 exposed repeated work: every HashMap code report allocates a key list and sorts it. At 10,000 items it used 5,256,240 ns per report. The limiting characteristic is unordered storage, causing repeated `O(n log n)` sorting. Objective: preserve average constant-time lookup, eliminate report-time sorting, and achieve at least 3x faster repeated reports at 20,000 items.

C-D. Revised design and prediction

`IndexedInventory` maintains three coordinated views: HashMap for operational lookup, TreeMap for code order, and LinkedHashMap for insertion order. `put` updates all three; reports iterate the appropriate index. This is an indexing redesign, not a simple collection swap. It trades extra write work and roughly three entry/reference sets for `O(n)` reports without temporary sort lists. Prediction: insertion becomes slower, reports become much faster, and benefit grows with size/report frequency.

E-F. Validation and defence

Medicines	Reports	Original report ns	Redesigned report ns	Speedup
1,000	100	48,822,000	13,062,400	3.74x
5,000	100	150,684,000	22,397,600	6.73x
20,000	100	604,057,400	48,513,300	12.45x

At 20,000, insertion increased from 175,052,800 to 190,390,700 ns, while reports improved 12.45x and exceeded the objective. The prediction was supported. The redesigned system is justified for frequent reports and multiple required orders. The original single HashMap remains preferable for small inventories, rare reports, or strict memory limits.

Conclusion

The experiments support decisions based on workload rather than collection names: Builder for mutable per-ticket text; ArrayList for random-access playlists and ArrayDeque for two-ended queues; immutable, contract-consistent

hash keys; explicit map views for competing order requirements; generic comparator chains for reusable rankings; and priority plus arrival ordering for triage. The Q7 redesign demonstrates that avoiding repeated work can be more important than a single operation's nominal complexity.

Appendix - Execution evidence

Screenshots below were rendered from the retained raw outputs after successful Java 17 execution.

Q1 evidence

Q1 - Ticket text benchmark

```
workload,method,avg_nanos,avg_memory_bytes,estimated_intermediate_objects
short_1x10,StringPlus,99580,33593,30
short_1x10,StringBuilder,31440,41984,2
short_1x10,StringBuffer,31960,41992,2
server_1000x50,StringPlus,3563000,6297249,150000
server_1000x50,StringBuilder,3056400,1363736,2000
server_1000x50,StringBuffer,4739460,1358936,2000
wide_200x200,StringPlus,13147440,20979932,120000
wide_200x200,StringBuilder,2593200,2034561,400
wide_200x200,StringBuffer,3616780,2022376,400
```

Q2 evidence

Q2 - Playlist benchmark

```
size,list,add_end_ns,front_churn_ns,middle_churn_ns,get_middle_ns,iterate_per_element_ns
1000,ArrayList,154,2215,878,149,142
1000,LinkedList,159,990,2956,1147,126
10000,ArrayList,77,2616,1406,141,36
10000,LinkedList,23,218,20863,10403,19
50000,ArrayList,21,13447,6607,135,69
50000,LinkedList,16,218,105038,53649,5

size,changed_condition,avg_ns_per_end_operation
1000,ArrayDequeUpNext,129
10000,ArrayDequeUpNext,88
50000,ArrayDequeUpNext,15
```

Q3 evidence

Q3 - Hash contract tests

```
Attendance completed-code output:
Unique students: 2

Investigation:
Test1 predicted=1 actual=1
Test2 predicted=2 actual=2
Test3 predicted_contains=false actual=false
Corrected predicted=1 actual=1

Before TODO 1, two distinct Student objects with the same registration number may occupy different identity-hash buckets, so the observed size is typica
```

Q4 evidence

Q4 - Pharmacy map benchmark

```
medicines,map,lookup_avg_ns,code_report_avg_ns,iteration_report_avg_ns,natural_code_order,natural_insertion_order
100,HashMap,85,465080,52920,false,false
100,TreeMap,144,72780,44460,true,false
100,LinkedHashMap,32,104060,64280,false,true
1000,HashMap,37,1171520,165900,false,false
1000,TreeMap,178,164620,157200,true,false
1000,LinkedHashMap,37,306160,136700,false,true
10000,HashMap,50,5256240,1399800,false,false
10000,TreeMap,285,681380,500480,true,false
10000,LinkedHashMap,85,1057460,392800,false,true
```

Q5 evidence

Q5 - Generic ranking tests

```
Completed-code output:
[Eagles(9,6), Bisons(9,4), Panthers(9,4), Cobras(6,1)]

Correct ties: [Ranked[name=Beta, points=10, goalDiff=5], Ranked[name=Alpha, points=10, goalDiff=2], Ranked[name=Zulu, points=10, goalDiff=2], Ranked[nam
Faulty overflow compare(MAX,MIN)=1
Correct compare(MAX,MIN)=-1
Tie resolved alphabetically=true
```

Q6 evidence

Q6 - Triage correctness and growth

```
Completed-code output:
Meena(S1) Kumar(S1) Lata(S2) Ravi(S3) John(S3) Asha(S5)

hand_order=P1 P3 P5 P7 P0 P6 P2 P4
actual_order=P1 P3 P5 P7 P0 P6 P2 P4
size,admit_avg_ns,serve_avg_ns
1000,1768,882
5000,404,429
20000,186,419
fair_rule=effective severity improves by one after each 30 waiting minutes; rebuild before selection
```

Q7 evidence

Q7 - Original versus redesigned

```
medicines,reports,original_insert_ns,redesigned_insert_ns,original_reports_ns,redesigned_reports_ns,speedup
1000,100,85053300,25453100,48822000,13062400,3.74
5000,100,44017100,51075800,150684000,22397600,6.73
20000,100,175052800,190390700,604057400,48513300,12.45
```